



Tentamen i Programmeringsteknik 1999-08-27, kl 8.00-13.00

Anvisningar

Hjälpmedel: Holm, Objektorienterad programmering och Java, samt Java-snabbpreferens.

Skrivningen innehåller tre uppgifter. Du bör behandla samtliga uppgifter.

Obs. Senast 3:e arbetsdagen efter tentamen anslås på anslagstavlan vilka som deltagit i tentamen men som, enligt institutionens noteringar, inte uppfyller tentamensvillkoren (alla 18 övningarna och de 3 inlämningsuppgifterna godkända). Deras skrivningar annulleras den 6:e arbetsdagen efter tentamen, såvida inte rättelse skett innan dess eller ansvarig lärare lämnat dispens.

Närmaste tentamenstillfällen i kursen:

Måndag 6 mars 2000, kl 8-13

Onsdag 24 maj 2000, kl 8-13

Något om bedömningen

- Tänk på att du med dina lösningar vänder dig till en mänsklig rättare, inte till en dator.
- Du kan förutsätta att rättaren är väl insatt i uppgifterna.
- Rättaren fäster ingen större uppmärksamhet vid fel som en kompilator enkelt skulle upptäcka.
- Det är viktigt att du gör lämpliga och tydliga indragningar och att du använder vettiga variabelnamn så att din lösning går lätt att följa. En god handstil är ett plus.
- Tvekar du mellan att presentera en smart lösning eller en lösning som är enkel att följa, välj den enklare versionen.
- När det gäller betyget 3 så har det ingen betydelse om någon lösning är onödigt ineffektiv.

Uppgift 1

”Erathostenes såll” är en algoritm med vars hjälp man kan ta reda på alla primtal mindre än eller lika med ett givet tal n . Metoden bygger på att man i omgångar stryker de tal som är jämnt delbara med primtalen 2, 3, 5, 7, 11, osv.

Det går till på följande sätt:

1. Skapa en boolean-vektor med index $0..n$ och ge alla vektorelementen värdet true. Vektorelement med index k svarar mot talet k . Elementen med index 0 och 1 används inte.
2. Börja med talet $k=2$. Stryk alla efterföljande tal som är jämnt delbara med 2 genom att markera dem med false i vektorn.
3. Fortsätt med nästa tal k som inte är struket. Stryk alla efterföljande tal som är jämnt delbara med k genom att markera dem med false i vektorn.
4. Upprepa moment 3 tills $k*k > n$.
5. De tal som nu inte har strukits är primtal.

Exempel ($n=27$, strukna tal anges med F):

	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
($k=2$)	T	T	F	T	F	T	F	T	F	T	F	T	F	T	F	T	F	T	F	T	F	T	F	T	F	T
($k=3$)	T	T	F	T	F	T	F	F	F	T	F	T	F	F	F	T	F	T	F	F	F	T	F	T	F	F
($k=5$)	T	T	F	T	F	T	F	F	F	T	F	T	F	F	F	T	F	T	F	F	F	T	F	F	F	F

Skriv en main-metod som läser in ett heltal och skriver ut alla primtal som är mindre än eller lika med det inlästa talet. Erathostenes såll skall användas.

Uppgift 2

I ett system för bokning av tentamenssalar ingår klasserna Room och RoomRegister:

```
class Room {
    ...      //inför lämpliga attribut!

    /** skapa en sal med namnet name (exempel: "MA10")
        med platser för size tenterande. */
    public Room(String name, int size) {
        ...   //implementera!
    }

    /** tag reda på namnet på salen */
    public String getName() {
        ...   //implementera!
    }

    /** tag reda på antalet platser i salen */
    public int getSize() {
        ...   //implementera!
    }
}

class RoomRegister {
    ...      //inför lämpliga attribut!

    /** skapa en salsvektor med plats för högst maxNbr salar,
        numrerade 0,1,2, osv. */
    public RoomRegister(int maxNbr) {
        ...   //implementera!
    }

    /** lägg in en ny sal i salsvektorn, salarna numreras i den ordning
        de läggs in, den första salen får nummer 0. */
    public void add(Room room) {
        ...   //implementera!
    }

    /** tag reda på antalet salar som finns i salsvektorn */
    public int getNbrOfRooms() {
        ...   //implementera!
    }

    /** tag reda på sal nummer roomNbr (salen förutsätts existera) */
    public Room getRoom(int roomNbr) {
        ...   //implementera!
    }
}
```

Bestäm lämpliga attribut för klasserna Room och RoomRegister och implementera konstruktorena och alla operationer.

Uppgift 3

I systemet för bokning av tentamenssalar som beskrivs i uppgift 2 finns också en klass `TimeInterval` med följande specifikation:

```
specification TimeInterval {
    /** skapa ett tidsintervall under dagen date med start klockan startHour och
        slut klockan finishHour (hela timmar) */
    TimeInterval(int date, int startHour, int finishHour);

    /** hämta dag för tidsintervallet */
    int getDate();

    /** hämta starttimme för tidsintervallet */
    int getStartHour();

    /** hämta sluttimme för tidsintervallet */
    int getFinishHour();

    /** tag reda på om tidsintervallet other "kolliderar" med det
        aktuella tidsintervallet */
    boolean overlaps(TimeInterval other);
}
```

Dessutom ingår en klass `Booking` med superklassen `Link`. Avsikten är att alla objekt av klassen `Booking`, dvs alla bokningar, läggs in i en lista av klassen `Head`. `Booking` har följande specifikation:

```
specification Booking extends Link {
    /** skapa en bokning av salen med nummer roomNbr under tidsintervallet time,
        course anger vilken kurs som skall tenteras */
    Booking(int roomNbr, TimeInterval time, String course);

    /** tag reda på salsnumret för bokningen */
    public int getRoomNbr();

    /** tag reda på tiden för bokningen */
    public TimeInterval getTime();

    /** tag reda på vilken kurs bokningen avser */
    public String getCourse();
}
```

Du skall utnyttja klasserna `TimeInterval` och `Booking` i operationerna `getFreeRoom` och `makeBooking` som finns i följande klass `BookingRegister`:

```
class BookingRegister {
    private RoomRegister roomRegister;
    private Head bookings; // lista med hittills gjorda bokningar

    /** skapa ett bokningsregister för salsregistret roomRegister */
    public BookingRegister(RoomRegister roomRegister) {
        this.roomRegister = roomRegister;
        this.bookings = new Head();
    }

    /** leta reda på en sal som är ledig under tiden time och som rymmer size
        tenterande (se även "Frivilligt tillägg" under uppgift a nästa sida!) */
    public int getFreeRoom(TimeInterval time, int size) {
        ... //implementera!
    }

    /** lägg in en ny bokning av salen med nummer roomNbr vid tiden time för
        kursen course (det förutsätts att salen är ledig) */
    public void makeBooking(int roomNbr, TimeInterval time, String course) {
        ... //implementera!
    }
}
```

(uppgift 3 fortsätter på nästa sida)

Uppgift 3 (forts)

a) Implementera operationen

```
public int getFreeRoom(TimeInterval time, int size)
```

Använd följande metod (observera att getFreeRoom finns i klassen BookingRegister och därmed har direkt tillgång till attributen roomRegister och bookings):

1. Inför en hjälpvektor booked med ett boolean-värde för varje sal (alla värden sätts till false från början, true anger att salen är bokad vid tiden time).
2. Gå igenom alla bokningar i listan bookings. Om en bokning av en sal kolliderar med den aktuella tiden time (använd operationen overlaps i TimeInterval) så markera salen som bokad vid tiden time (genom att sätta motsvarande element i booked-vektorn till true).
3. Gå igenom salarna i roomRegister och finn en ledig sal för minst size tenderande.
4. Om en ledig sal med tillräcklig storlek påträffas skall operationen getFreeRoom returnera salens nummer, annars värdet -1.

Frivilligt tillägg i första hand för dem som eftersträvar betyg 4 eller 5:

Välj i moment 3 ovan den *minsta* lediga salen som rymmer size tenderande.

b) Implementera operationen

```
public void makeBooking(int roomNbr, TimeInterval time, String course)
```

Exempel på uppbyggnaden av ett salsregister med bokningsregister

I nedanstående figur visas ett salsregister med tre salar: MA8, MA9 och MA10. Bokningsregistret visar att MA9 är bokad för PfE 990827 kl 8-13, MA10 för Analys 1 990819 kl 8-13 och för Analys 2 990823 kl 8-13.

